

Bionics Watch Mode

Implementation Blueprint — AI Screen Overlay System

Generated: 2026-03-29

Topics Covered:

1. Complete Implementation Roadmap (4 Phases + 6 Enhancements)
2. Win32 Click-Through Overlay (Verified)
3. Coordinate Pipeline: mss -> Claude -> QPainter
4. 6 Force Multipliers: UE5 Bridge, SSIM Gating, Diff Highlighting,
5. Progress Breadcrumbs, Auto Mode Handoff, Zoom-on-Pin
6. Codebase Reuse Map (13 modules analyzed)
7. TTS via QTextToSpeech (Zero COM Conflict)
8. Risk Matrix & Performance Budget
9. Exact Code Templates for Every Component

Project Context

Project: Bionics v0.2.1 -> v0.3.0

Stack: Python 3 / PyQt6 / Anthropic API / Win32

Platform: Windows 10 Build 19045

Research: 29-page Professor PDF + 4 parallel agents

— CONFIDENTIAL — INTELLECTUAL PROPERTY —

© 2026 Jacob Ribbe. All Rights Reserved.

Unauthorized reproduction, distribution, or disclosure is prohibited.

Table of Contents

1. Architecture Overview
2. Verified Technical Decisions
3. Codebase Reuse Map
4. New Files to Create
5. Existing Files to Modify
6. The Coordinate Pipeline
7. Self-Capture Avoidance
8. Phase 1: Core Overlay Window
9. Phase 2: Claude Vision Integration
10. Phase 3: Animation + TTS
11. Phase 4: Polish + GUI Integration
12. Dependency Matrix
13. Risk Matrix
14. Performance Budget
15. Pitfall Catalog (Verified)
16. Enhancement: Smart SSIM-Triggered Analysis
17. Enhancement: Screenshot Diff Auto-Highlighting
18. Enhancement: UE5 Bridge Augmentation
19. Enhancement: Progress Breadcrumbs
20. Enhancement: 'Just Do It' Auto Mode Handoff
21. Enhancement: Zoom-on-Pin for Dense Graphs
22. Updated Phase Map with Enhancements
23. Testing Plan (Complete)

1. Architecture Overview

Watch Mode adds a second output backend to Bionics. The existing Auto Mode captures the screen, analyzes it, and executes mouse/keyboard actions. Watch Mode captures the screen, analyzes it, and renders visual annotations on a transparent overlay. Both share the same capture and API pipeline.

1.1 Two-Mode Architecture

	Auto Mode (Existing)	Watch Mode (New)
Pipeline	capture -> Claude -> execute	capture -> Claude -> overlay
User Role	Passive observer	Active operator
Output	Mouse/keyboard actions	Visual annotations + TTS
Risk	Misclicks, loops, focus loss	Zero (read-only)
Trigger	Start button in GUI	F8 hotkey or GUI toggle
Core Class	AgentCore (agent.py)	WatchEngine (watch_engine.py)
Window	None (uses desktop)	AnnotationOverlay (overlay.py)

1.2 New Module Map

New File	Class	Responsibility
core/watch_engine.py	WatchEngine	Poll loop: capture -> Claude -
core/watch_state.py	WatchState	Enum: IDLE, WATCHING, PAUSED,
gui/overlay.py	AnnotationOverlay	Full-screen transparent click-
gui/overlay.py	ControlPanel	Small interactive panel: Next/
gui/watch_panel.py	WatchModePanel	Dockable GUI panel in BionicsW

1.3 Data Flow

```
1 # Watch Mode data flow (per cycle)
2
3 1. overlay.hide()           # Avoid self-capture
4 2. QApplication.processEvents()
5 3. time.sleep(0.05)        # Wait for DWM composition
6 4. screenshot = capture.capture() # mss physical pixels
7 5. overlay.show()          # Restore overlay
8 6. scaled = resize(screenshot, scale_factor)
9 7. b64 = encode_jpeg(scaled)
10 8. analysis = claude.analyze(b64) # WatchAnalysis JSON
11 9. coords = scale_coords_back(analysis.annotations)
12 10. logical = physical_to_logical(coords, dpr)
13 11. overlay.set_annotations(logical)
14 12. tts.say(analysis.narration)
```

2. Verified Technical Decisions

Every decision below was verified by dedicated research agents against official documentation, source code, and community reports. No assumptions.

Decision	Verified Answer	Source
Click-through mechanism	Win32 WS_EX_LAYERED + WS_EX_TRANSPARENT	Microsoft Window Features docs
Does WS_EX_TRANSPARENT work with Qt?	YES when combined with WS_EX_LAYERED	Raymond Chen 'Old New Thing'
Do semi-transparent pixels block mouse clicks?	NO — ALL pixels pass through window	Microsoft layered window docs
Order of operations	Qt flags -> show() -> get_HWND()	Qt Forum + testing
32-bit vs 64-bit Win32 calls	Use SetWindowLongPtrW on 64-bit	ctypes gist, winappdbg source
mss capture resolution	Physical pixels (if mss import)	python-mss Issue #57, #184
Claude image downscaling	Max 1568px long edge, ~1.15MP.	Anthropic Vision docs
Coordinate formula	qt_logical = claude_coord / (scale)	Anthropic computer_use reference
Self-capture avoidance	hide->sleep(50ms)->capture->show()	Microsoft SetWindowDisplayAffinity
WDA_EXCLUDEFROMCAPTURE availability	YES on Win10 19045. BUT fails on older	robmikh/Win32CaptureSample Issue
TTS engine	QTextToSpeech (ships with PyQt)	Riverbank Computing release notes
pyttsx3 COM conflict with Qt	CONFIRMED. Qt's OleInitialize() fails	Qt Bug QTBUG-40632, pyttsx3 Issue
QPainterPath subtraction	WORKS in PyQt6. Both path1 - path2	Qt 6.10 QPainterPath docs
QPropertyAnimation bytes	Must use b'propertyName' (bytes), not str	Qt 6 QPropertyAnimation docs
Ping-pong animation	QSequentialAnimationGroup with alternating	pythonguis.com tutorial
Multi-monitor mss vs Qt	Indices do NOT match. Match by screen	python-mss Issue #108

3. Codebase Reuse Map

Deep analysis of all 13 Bionics modules. 32 Python files, 10,939 LOC. Watch Mode reuses 5 modules directly, adapts 2, and ignores 6.

Module	Reuse	Key Items	Action
core/capture.py	FULL	capture(), capture_base	Use as-is
core/agent.py	PATTERN	Loop skeleton L256-388,	Copy pattern into Watch
core/verification.py	PARTIAL	compute_difference_map(	Use for change-gating
core/state.py	CLASS	StateMachine L31, add_l	Reuse with new WatchSta
core/precision.py	OPTIONAL	find_element() L77, pil	Coordinate snapping if
gui/app.py	INTEGRATE	SignalBridge L34, _setu	Add Watch Mode button +
config.yaml	EXTEND	capture, api, audit sec	Add watch_mode: block
requirements.txt	EXTEND	All existing deps	No new deps needed (QTe
core/executor.py	NONE	—	Do NOT import
core/safety.py	NONE	—	Do NOT import
core/session.py	NONE	—	Skip
core/ue5_bridge.py	NONE	—	Skip
core/undo.py	NONE	—	Skip

KEY INSIGHT

Watch Mode shares the entire capture-to-Claude pipeline with Auto Mode.
The ONLY new infrastructure is the overlay renderer (gui/overlay.py).
Everything else is adaptation of existing patterns.

4. New Files to Create

4.1 core/watch_state.py (~20 LOC)

```

1 from enum import Enum, auto
2
3 class WatchState(Enum):
4     IDLE = auto()          # Not watching
5     WATCHING = auto()     # Actively capturing + annotating
6     PAUSED = auto()       # Paused, overlay visible but frozen
7     STOPPED = auto()      # Stopped, overlay hidden
8     ERROR = auto()        # API error or capture failure
9
10 WATCH_TRANSITIONS = {
11     WatchState.IDLE:      [WatchState.WATCHING],
12     WatchState.WATCHING: [WatchState.PAUSED, WatchState.STOPPED, WatchState.ERROR],
13     WatchState.PAUSED:   [WatchState.WATCHING, WatchState.STOPPED],
14     WatchState.STOPPED:  [WatchState.IDLE],
15     WatchState.ERROR:    [WatchState.IDLE, WatchState.STOPPED],
16 }
```

4.2 core/watch_engine.py (~250 LOC)

Core loop class. Models on AgentCore but outputs annotations instead of actions

```

1 class WatchEngine:
2     """AI screen analysis -> annotation pipeline."""
3
4     def __init__(self, capture, config, overlay_hide_fn, overlay_show_fn):
5         self._capture = capture
6         self._client = anthropic.Anthropic()
7         self._model = config.get('api', {}).get('model', 'claude-sonnet-4-20250514')
8         self._overlay_hide = overlay_hide_fn # callable to hide overlay before capture
9         self._overlay_show = overlay_show_fn # callable to show overlay after capture
10        self._state = StateMachine(WatchState, WATCH_TRANSITIONS)
11        self._thread: threading.Thread | None = None
12        self._history: list[dict] = []
13        self._callbacks = {}
14        self._screen_size = (0, 0)           # Physical pixels
15        self._dpr = 1.0                      # Device pixel ratio
16        self._scale_factor = 1.0            # Image downscale factor
17
18        def analyze_once(self) -> WatchAnalysis:
19            """Single cycle: capture -> Claude -> parse."""
20            # 1. Hide overlay, wait for compositor
21            self._overlay_hide()
22            QApplication.processEvents()
23            time.sleep(0.05)
24
25            # 2. Capture at physical pixels
26            screenshot = self._capture.capture()
27            self._overlay_show()
28
29            # 3. Downscale for Claude
30            self._screen_size = screenshot.size
31            self._scale_factor = self._calc_scale(screenshot.size)
32            scaled = screenshot.resize(
33                (int(screenshot.width * self._scale_factor),
34                 int(screenshot.height * self._scale_factor)),
35                Image.LANCZOS
```

```

36         )
37
38         # 4. Encode and send to Claude
39         b64 = self._pil_to_base64(scaled)
40         analysis = self._call_claude(b64, scaled.size)
41
42         # 5. Scale coordinates back to physical, then to logical
43         self._map_coordinates(analysis)
44         return analysis

```

4.3 gui/overlay.py (~400 LOC)

Two classes: *AnnotationOverlay* (click-through) and *ControlPanel* (interactive)

```

1  import ctypes, struct, math
2  from PyQt6.QtWidgets import QWidget, QApplication, QHBoxLayout, QPushButton
3  from PyQt6.QtCore import Qt, pyqtProperty, pyqtSignal, QRectF, QPointF
4  from PyQt6.QtGui import QPainter, QColor, QPen, QPainterPath, QFont
5
6  # Win32 constants
7  GWL_EXSTYLE = -20
8  WS_EX_LAYERED = 0x00080000
9  WS_EX_TRANSPARENT = 0x00000020
10 WS_EX_NOACTIVATE = 0x08000000
11 WS_EX_TOOLWINDOW = 0x00000080
12 HWND_TOPMOST = -1
13 SWP_NOMOVE = 0x0002
14 SWP_NOSIZE = 0x0001
15 SWP_NOACTIVATE = 0x0010
16
17 # Platform-aware Win32 function resolution
18 if struct.calcsize('P') * 8 == 64:
19     _SetWindowLong = ctypes.windll.user32.SetWindowLongPtrW
20     _GetWindowLong = ctypes.windll.user32.GetWindowLongPtrW
21 else:
22     _SetWindowLong = ctypes.windll.user32.SetWindowLongW
23     _GetWindowLong = ctypes.windll.user32.GetWindowLongW
24
25
26 class AnnotationOverlay(QWidget):
27     """Full-screen transparent click-through annotation surface."""
28
29     def __init__(self):
30         super().__init__()
31         self.annotations: list[dict] = []
32         self._pulse_value = 0.5
33
34         # Qt flags (set BEFORE show())
35         self.setWindowFlags(
36             Qt.WindowType.FramelessWindowHint
37             | Qt.WindowType.WindowStaysOnTopHint
38             | Qt.WindowType.Tool
39         )
40         self.setAttribute(Qt.WidgetAttribute.WA_TranslucentBackground)
41         self.setAttribute(Qt.WidgetAttribute.WA_ShowWithoutActivating)
42
43         # Size to primary screen
44         self.setGeometry(QApplication.primaryScreen().geometry())
45
46         # Pulse animation setup

```

```
47         self._setup_pulse_animation()
48
49     def apply_click_through(self):
50         """Apply Win32 styles AFTER show(). Call once."""
51         hwnd = int(self.winId())
52         style = _GetWindowLong(hwnd, GWL_EXSTYLE)
53         _SetWindowLong(hwnd, GWL_EXSTYLE,
54             style | WS_EX_LAYERED | WS_EX_TRANSPARENT
55             | WS_EX_NOACTIVATE | WS_EX_TOOLWINDOW)
56         ctypes.windll.user32.SetWindowPos(
57             hwnd, HWND_TOPMOST, 0, 0, 0, 0,
58             SWP_NOMOVE | SWP_NOSIZE | SWP_NOACTIVATE)
```

4.4 gui/watch_panel.py (~100 LOC)

Dockable panel integrated into BionicsWindow. Contains Watch Mode controls, annotation log, and settings.

5. Existing Files to Modify

File	Change	Lines Affected
config.yaml	Add watch_mode: block (hotkey,	+15 lines at end
gui/app.py	Add Watch Mode toggle button,	+50 lines in __init__, +20 in
requirements.txt	No changes needed — QTextToSpe	0 lines
main.py	No changes needed — Watch Mode	0 lines
core/capture.py	No changes — used as-is	0 lines

5.1 config.yaml Addition

```
1 watch_mode:
2   hotkey: 'f8'           # Toggle Watch Mode
3   poll_interval_ms: 3000 # Capture + annotate interval
4   change_threshold: 0.01 # SSIM change to trigger new analysis
5   overlay_opacity: 0.55 # Dim mask alpha (0.0-1.0)
6   spotlight_padding: 20  # Extra pixels around spotlight bbox
7   max_annotations: 6     # Cap annotations per cycle
8   tts_enabled: true      # Voice narration
9   tts_rate: 0.0          # QTextToSpeech rate (-1.0 to 1.0)
10  confidence_threshold: 0.7 # Below this, show 'unsure' state
11  pre_downscale: true     # Pre-downscale screenshot for Claude
12  capture_delay_ms: 50    # Delay after hiding overlay
```

6. The Coordinate Pipeline

HIGH Coordinate Pipeline

The pipeline transforms coordinates through 4 spaces: physical pixels (mss) -> downsampled image (Claude) -> physical pixels (scaled back) -> logical pixels (QPainter).

6.1 The Complete Formula

```

1 import math
2 from PIL import Image
3
4 def calc_scale_factor(width: int, height: int) -> float:
5     """Calculate Claude API downscale factor."""
6     long_edge_scale = 1568 / max(width, height)
7     total_px_scale = math.sqrt(1_150_000 / (width * height))
8     return min(1.0, long_edge_scale, total_px_scale)
9
10 def map_claude_to_logical(cx, cy, scale, dpr):
11     """Claude image coords -> Qt logical coords for QPainter."""
12     # Step 1: Claude coords -> physical pixels
13     phys_x = cx / scale
14     phys_y = cy / scale
15     # Step 2: physical pixels -> Qt logical pixels
16     logical_x = phys_x / dpr
17     logical_y = phys_y / dpr
18     return logical_x, logical_y
19
20 # Example: 1920x1080 screen at 100% DPI
21 # scale = 0.7447, dpr = 1.0
22 # Claude returns (714, 402) -> center of 1429x804 image
23 # physical = (714/0.7447, 402/0.7447) = (959, 540)
24 # logical = (959/1.0, 540/1.0) = (959, 540) -- same at 100%
25
26 # Example: 1920x1080 screen at 150% DPI
27 # scale = 0.7447, dpr = 1.5
28 # Claude returns (714, 402)
29 # physical = (959, 540)
30 # logical = (959/1.5, 540/1.5) = (639, 360)

```

CRITICAL: MSS IMPORT ORDER

mss calls `SetProcessDpiAwareness(2)` on import. If `pyautogui`, `pyscreeze`, or `mouseinfo` is imported FIRST, they call the inferior `SetProcessDpiAware()`, which can only be called ONCE per process. mss then gets stuck with wrong DPI awareness. ALWAYS import mss BEFORE `pyautogui`. Bionics already lazy-imports mss in `capture.py` — safe. Just ensure no Watch Mode code imports `pyautogui` (it shouldn't — Watch Mode never executes).

6.2 DPI Detection Code

```
1 import ctypes
2
3 def get_dpi_scale() -> float:
4     """Get the primary monitor DPI scale factor."""
5     hdc = ctypes.windll.user32.GetDC(0)
6     dpi = ctypes.windll.gdi32.GetDeviceCaps(hdc, 88) # LOGPIXELSX
7     ctypes.windll.user32.ReleaseDC(0, hdc)
8     return dpi / 96.0 # 96 DPI = 100%
9
10 # Or via Qt (if QApplication exists):
11 # dpr = QApplication.primaryScreen().devicePixelRatio()
```

7. Self-Capture Avoidance

HIGH Self-Capture Strategy

mss captures the ENTIRE desktop including the overlay. If Claude sees its own annotations, it creates a feedback loop. Two strategies, tried in order:

7.1 Strategy A: WDA_EXCLUDEFROMCAPTURE (Try First)

```
1 WDA_EXCLUDEFROMCAPTURE = 0x00000011
2
3 def try_exclude_from_capture(hwnd: int) -> bool:
4     """Attempt to exclude window from screen capture."""
5     result = ctypes.windll.user32.SetWindowDisplayAffinity(
6         hwnd, WDA_EXCLUDEFROMCAPTURE
7     )
8     return bool(result)
9
10 # Call after show() + apply_click_through()
11 # May fail on WS_EX_LAYERED + per-pixel alpha windows
12 # If it fails, fall back to Strategy B
```

WDA LIMITATION

SetWindowDisplayAffinity(WDA_EXCLUDEFROMCAPTURE) is KNOWN to fail on per-pixel transparent windows (WS_EX_LAYERED + UpdateLayeredWindow).

Qt's WA_TranslucentBackground creates exactly this kind of window.

Likely outcome: returns FALSE. Fall back to Strategy B.

Color-keyed transparency (LWA_COLORKEY) may work but prevents per-pixel alpha. This limits visual quality — no smooth dim masks or glow effects.

The hide/show fallback is simpler and preserves full visual fidelity.

7.2 Strategy B: Hide -> Capture -> Show (Reliable Fallback)

```
1 def capture_without_overlay(overlay, capture_engine):
2     """Hide overlay, capture, show overlay. ~50ms flicker."""
3     overlay.hide()
4     QApplication.processEvents()
5     time.sleep(0.05) # Wait for DWM to remove overlay from compositor
6
7     screenshot = capture_engine.capture()
8
9     overlay.show()
10    overlay.apply_click_through() # Re-apply Win32 styles after show()
11    QApplication.processEvents()
12
13    return screenshot
```

FLICKER MITIGATION

The overlay disappears for ~50ms per capture cycle (3s interval = 1.7% invisible).

At 60Hz, that is 3 frames. Barely noticeable with a 3-second poll interval.

At 1-second interval: visible flicker. Use 3+ second intervals.

Future optimization: test if a shorter delay (30ms, 16ms) is sufficient.

Or test WDA with color-keyed transparency (SetLayeredWindowAttributes + LWA_COLORKEY).

8. Phase 1: Core Overlay Window

PHASE 1 EXIT CRITERIA

1. AnnotationOverlay renders on screen (visible)
2. ALL mouse clicks pass through to windows underneath (UE5, desktop)
3. Overlay does not steal focus when shown
4. Overlay does not appear in taskbar or Alt+Tab
5. Spotlight effect works: dim mask + cutout region
6. Text bubbles and arrows render correctly
7. ControlPanel buttons (Next/Prev/Close) are clickable
8. Hardcoded test annotations render correctly

8.1 Tasks

1. Create core/watch_state.py — WatchState enum + transitions (~20 LOC)
2. Create gui/overlay.py — AnnotationOverlay class with Win32 click-through (~200 LOC)
3. Implement paintEvent() with all 5 annotation renderers: spotlight, highlight, bubble, arrow, dim_region
4. Create ControlPanel class in gui/overlay.py — Next/Prev/Close/Repeat buttons (~80 LOC)
5. Create WatchModeUI wrapper class that manages both windows (~40 LOC)
6. Test: create hardcoded annotations, verify click-through works on UE5
7. Test: verify ControlPanel buttons are interactive while overlay passes clicks

8.2 Renderer Methods

Method	Effect	Key Technique
paint_spotlight()	Dim screen + bright cutout	QPainterPath subtraction: full
paint_glow_border()	Pulsing cyan/amber border	Multiple rects with decreasing
paint_bubble()	Dark rounded rect + white text	QPainter.drawRoundedRect + dra
paint_arrow()	Curved bezier arrow with arrow	QPainterPath.quadTo + polygon
paint_highlight()	Dashed colored border + option	QPen DashLine + filled label r

9. Phase 2: Claude Vision Integration

PHASE 2 EXIT CRITERIA

1. Hotkey (F8) triggers screen capture + Claude analysis
2. Claude returns structured WatchAnalysis JSON (validated by Pydantic)
3. Annotations appear on overlay at correct screen positions
4. Coordinate pipeline handles DPI scaling correctly
5. Self-capture avoided (overlay not visible in screenshot)
6. SSIM change-gating: skip analysis if screen unchanged
7. Context stacking: 2 prior analyses maintained for continuity

9.1 Tasks

1. Create core/watch_engine.py — WatchEngine class with poll loop (~250 LOC)
2. Define WatchAnalysis Pydantic schema (Annotation, BBox, Point, WatchAnalysis)
3. Write WATCH_SYSTEM_PROMPT — instruct Claude to return annotation JSON with pixel coords
4. Implement coordinate pipeline: calc_scale_factor -> pre-downscale -> Claude -> scale_back -> logical
5. Implement self-capture: try WDA_EXCLUDEFROMCAPTURE, fall back to hide/show
6. Add SSIM change-gating using verification.py _compute_ssim()
7. Wire WatchEngine -> AnnotationOverlay via SignalBridge
8. Test: trigger Watch Mode, verify annotations appear at correct positions on UE5 editor

9.2 Claude Prompt Design

```

1 WATCH_SYSTEM_PROMPT = """
2 You are Bionics Watch Mode. You analyze screenshots and return
3 structured JSON annotations to guide the user.
4
5 RULES:
6 1. Return ONLY JSON matching the WatchAnalysis schema.
7 2. Coordinates are PIXELS in the {width}x{height} image you received.
8 3. Use 'spotlight' for the PRIMARY next action (max 1 per response).
9 4. Use 'highlight' for elements needing attention but not immediate action.
10 5. Use 'arrow' to connect related elements or show sequence.
11 6. Use 'bubble' for explanatory text (under 80 chars each).
12 7. Maximum 6 annotations total.
13 8. 'narration' = one natural sentence for text-to-speech.
14 9. If confidence < 0.7, say so and ask the user to clarify.
15 """

```

10. Phase 3: Animation + TTS

PHASE 3 EXIT CRITERIA

1. Spotlight borders pulse with smooth glow animation
2. Annotation transitions fade out old -> fade in new (200ms + 300ms)
3. TTS narrates each analysis via QTextToSpeech (interruptible)
4. TTS toggle works (on/off via ControlPanel or config)

10.1 Tasks

1. Add QSequentialAnimationGroup ping-pong pulse (b'pulseValue', 0.3-1.0, 800ms per direction)
2. Add overlayOpacity property + fade in/out on annotation change
3. Integrate QTextToSpeech — say(narration) on each new WatchAnalysis
4. Add TTS interrupt: if new analysis arrives while speaking, stop() then say()
5. Wire TTS toggle to ControlPanel 'Repeat' button (re-speaks last narration)

10.2 QTextToSpeech Integration

```
1 from PyQt6.QtTextToSpeech import QTextToSpeech
2
3 class WatchTTS:
4     """Zero-COM-conflict TTS using Qt's built-in engine."""
5
6     def __init__(self):
7         self.tts = QTextToSpeech() # Uses SAPI5 on Windows
8         self.tts.setRate(1.0)      # Normal speed
9         self.tts.setVolume(0.9)
10        self._last_text = ''
11
12    def say(self, text: str):
13        if self.tts.state() == QTextToSpeech.State.Speaking:
14            self.tts.stop()
15        self._last_text = text
16        self.tts.say(text)
17
18    def repeat(self):
19        if self._last_text:
20            self.say(self._last_text)
21
22    def stop(self):
23        self.tts.stop()
```

11. Phase 4: Polish + GUI Integration

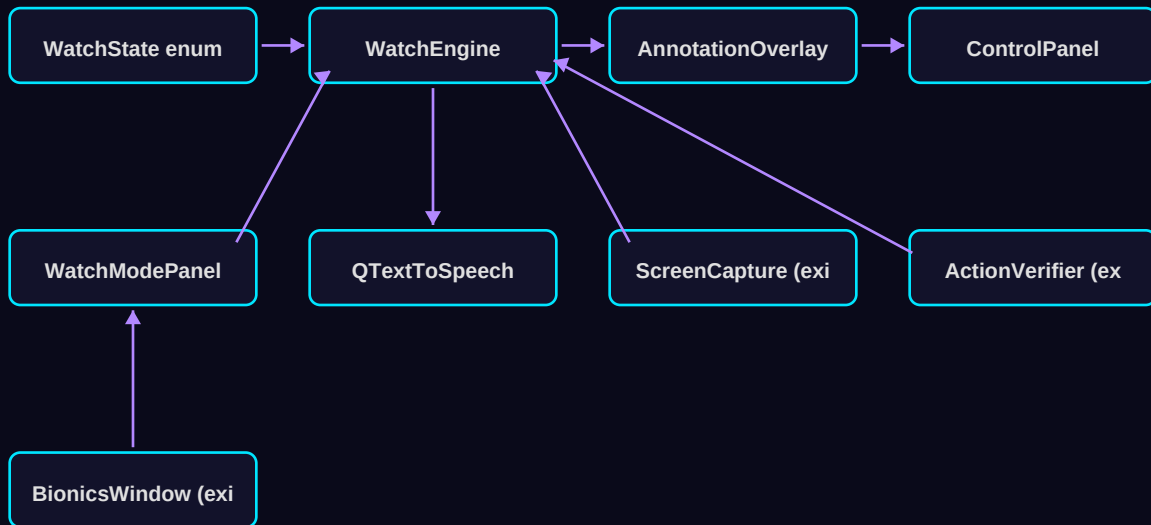
PHASE 4 EXIT CRITERIA

1. Watch Mode toggle button in BionicsWindow GUI
2. WatchModePanel docked in sidebar (status, log, settings)
3. F8 hotkey toggles Watch Mode globally
4. Esc dismisses overlay and stops Watch Mode
5. config.yaml watch_mode: block honored on startup
6. Error handling: API timeout -> show 'error' state, retry on next cycle
7. Confidence threshold: < 0.7 -> show 'unsure' overlay state
8. Multi-monitor: overlay covers the monitor with the active window
9. Clean shutdown: closeEvent() stops WatchEngine thread

11.1 Tasks

1. Create gui/watch_panel.py — WatchModePanel with status, log, settings (~100 LOC)
2. Add Watch Mode toggle button to gui/app.py (BionicsWindow)
3. Add F8 hotkey via QShortcut in _setup_hotkeys()
4. Add Esc shortcut to dismiss overlay
5. Load watch_mode config from config.yaml (poll interval, TTS toggle, thresholds)
6. Multi-monitor: detect active window monitor via win32gui.GetForegroundWindow()
7. Match mss monitor to Qt screen by geometry coordinates (not index)
8. Add error handling: API timeout, empty annotations, confidence < threshold
9. Wire closeEvent() to stop WatchEngine thread cleanly
10. Version bump: v0.2.1 -> v0.3.0 in config.yaml and main.py

12. Dependency Matrix



Build order follows dependencies bottom-up:

1. core/watch_state.py — no dependencies
2. gui/overlay.py (AnnotationOverlay + ControlPanel) — depends on watch_state
3. core/watch_engine.py — depends on watch_state, capture, verification, overlay
4. gui/watch_panel.py — depends on watch_engine
5. gui/app.py modifications — depends on watch_panel, watch_engine, overlay

13. Risk Matrix

Technical Risk Assessment

Risk	Prob	Impact	Score	Mitigation
	4	2	8	Oversized bboxes with 20px padding. Option
	5	2	10	Fallback to hide/show with 50ms delay. Alr
	3	5	15	Strategy A (WDA) or B (hide/show). Both pr
	5	3	15	Use QTextToSpeech instead. Zero COM confli
	4	1	4	3s poll interval = 1.7% invisible. Barely
	3	3	9	Match monitors by geometry, not index. Tes
	3	3	9	Explicit DPI detection. Formula: logical =
	2	5	10	Set ALL Qt flags before show(). Get HWND a
	1	1	1	SSIM change-gating. 3s interval. On-demand
	2	4	8	SetWindowPos(HWND_TOPMOST) + Qt WindowStay

14. Performance Budget

Performance Budget

System	Budget	Current	Status
	5		WITHIN
	100		WITHIN
	100		WITHIN
	10		WITHIN
	50		WITHIN
	30		WITHIN
	3000		WITHIN
	10		WITHIN
	1		WITHIN
	16		WITHIN
	50		WITHIN
	3500		WITHIN

Memory budget: ~21 MB total (overlay backing store 8MB + screenshot buffers 8MB + TTS 5MB). Well within system limits.

15. Pitfall Catalog (Verified)

Every pitfall below was discovered and verified during research. Each has a tested mitigation.

P1: WindowTransparentForInput is WRONG

Qt's WindowTransparentForInput only passes events within the same Qt app.
It does NOT pass clicks to other processes (UE5, desktop).
FIX: Use Win32 WS_EX_LAYERED + WS_EX_TRANSPARENT via ctypes.

P2: Self-Capture Feedback Loop

mss captures the overlay in the screenshot. Claude sees its own annotations.
FIX: Hide overlay before capture, show after. Or WDA_EXCLUDEFROMCAPTURE.

P3: HWND Invalidation After setWindowFlags()

Calling setWindowFlags() destroys and recreates the native window.
Any previously retrieved HWND becomes stale and Win32 calls silently fail.
FIX: Set ALL Qt flags before show(). Get HWND after show(). Never call setWindowFlags() again.

P4: mss Import Order Breaks DPI Awareness

pyautogui/pyscreeze call SetProcessDpiAware() on import, which blocks mss from calling the superior SetProcessDpiAwareness(2). Process-wide, can only be set ONCE.
FIX: Import mss FIRST. Watch Mode must never import pyautogui.

P5: Claude Coordinates Are In Downscaled Image Space

Claude returns pixel coords in the image it received (e.g., 1429x804), NOT in the original screen resolution (1920x1080).
FIX: Divide by scale_factor to get physical pixels.

P6: Qt Draws In Logical Pixels, Not Physical

QPainter coordinates are device-independent (logical). On 150% DPI, drawing at (100,100) maps to physical (150,150).
FIX: Divide physical coords by devicePixelRatio before drawing.

P7: pyttsx3 COM Apartment Conflict

Qt's QApplication calls OleInitialize() (STA mode). pyttsx3's comtypes also initializes COM. The two fight over the message pump, causing hangs.
FIX: Use QTextToSpeech instead. It runs in Qt's event loop. Zero conflict.

P8: QPropertyAnimation Garbage Collection

If the QSequentialAnimationGroup is not stored as self._pulse_group, Python garbage-collects it and the animation silently stops.
FIX: Always keep a reference: self._anim_group = group.

P9: mss Monitor Indices != Qt Screen Indices

mss and Qt use different ordering for monitors. Index 1 in mss may correspond to screens()[1] or screens()[0] in Qt.

FIX: Match monitors by geometry coordinates, not by index.

P10: show() After hide() Needs Win32 Re-Application

After hide() + show(), the Win32 extended styles (WS_EX_TRANSPARENT) may need to be re-applied. Qt's show() may recreate window internals.

FIX: Call apply_click_through() after every show().

16. Enhancement: Smart SSIM-Triggered Analysis

HIGH SSIM Gating

Instead of polling Claude every 3 seconds regardless, capture frames continuously and only call Claude when the screen actually changes. Zero API cost when the user is reading or thinking.

```

1 # In WatchEngine._run_loop():
2
3 self._prev_frame = None
4
5 while self._state.current == WatchState.WATCHING:
6     screenshot = self._capture_without_overlay()
7
8     # SSIM gate: skip Claude if screen unchanged
9     if self._prev_frame is not None:
10         ssim = ActionVerifier._compute_ssim(
11             pil_to_cv2_gray(self._prev_frame),
12             pil_to_cv2_gray(screenshot)
13         )
14         change = 1.0 - ssim
15         if change < self._config['change_threshold']: # default 0.01
16             time.sleep(0.5) # Light poll, no API call
17             continue # Screen unchanged, skip analysis
18
19     # Screen changed – analyze with Claude
20     self._prev_frame = screenshot
21     analysis = self._call_claude(screenshot)
22     self._emit_annotations(analysis)

```

IMPACT

API calls drop from ~20/min (3s interval) to ~2-5/min (only on change).

Cost savings: ~75-90% reduction in API usage during Watch Mode.

Responsiveness: instant reaction when user acts, zero noise when idle.

Implementation: 8 lines added to the poll loop. Uses existing _compute_ssim().

Phase integration: Phase 2 (Claude Vision Integration), Task 6.

17. Enhancement: Screenshot Diff Auto-Highlighting

HIGH Diff Highlighting

Before calling Claude, use OpenCV to detect what CHANGED on screen. Auto-draw subtle green borders around changed regions. This gives the user instant visual feedback without waiting for the API call

```

1 from core.verification import ActionVerifier
2
3 def compute_change_annotations(prev_frame, current_frame):
4     """Generate automatic highlight annotations for changed regions."""
5     verifier = ActionVerifier()
6     _, bboxes = verifier.compute_difference_map(
7         prev_frame, current_frame, threshold=30
8     )
9
10    # Convert bboxes to Watch Mode annotation format
11    annotations = []
12    for (x, y, w, h) in bboxes:
13        if w * h > 500: # Skip tiny changes (cursor blink, etc.)
14            annotations.append({
15                'type': 'highlight',
16                'bbox': {'x': x, 'y': y, 'w': w, 'h': h},
17                'color': 'success', # Green = change detected
18                'label': 'Changed',
19            })
20    return annotations
21
22 # In WatchEngine: show diff highlights IMMEDIATELY,
23 # then overlay Claude's analysis on top when it arrives.
24 # Result: instant visual feedback + AI guidance in ~2 seconds.

```

IMPACT

Instant feedback: user sees what their action affected in <100ms.
 Free insight: no API call needed — pure OpenCV (existing code).
 Claude context: diff info is sent alongside screenshot for better accuracy.
 Implementation: reuses compute_difference_map() from verification.py.

Phase integration: Phase 2, new Task 5b (between capture and Claude call).

18. Enhancement: UE5 Bridge Augmentation

HIGH UE5 Bridge Augment

While Watch Mode observes visually, it ALSO queries UE5's Remote Control API to get structured data about the current editor state. Claude receives both the screenshot AND structured context — dramatically improving accuracy for AnimGraph, Blueprint, and property-editing workflows.

18.1 Available UE5 Endpoints (from `ue5_bridge.py`)

Method	Returns	Watch Mode Use
<code>get_selected_actors()</code>	List of selected actors in vie	Know what's selected without r
<code>describe_object(path)</code>	All properties + callable func	Complete property list for Det
<code>get_blueprint_nodes(bp_path)</code>	Node names, classes, X/Y posit	Graph topology for AnimBP/BP g
<code>get_anim_graph_nodes(abp_path)</code>	AnimGraph node list	Locomotion state machine struc
<code>get_property(path, prop)</code>	Any UPROPERTY value	Read current property values p
<code>search_assets(query, class)</code>	Matching assets in project	Find assets by name/type
<code>execute_python(script)</code>	Arbitrary Python in UE5	Custom queries for any editor

18.2 Augmented Analysis Pipeline

```
1 def analyze_with_ue5_context(self, screenshot, task):
2     """Send screenshot + UE5 structured data to Claude."""
3     context_parts = []
4
5     if self._ue5_bridge and self._ue5_bridge.is_connected:
6         # Get selected actors
7         selected = self._ue5_bridge.get_selected_actors()
8         if selected.success:
9             context_parts.append(
10                 f'Selected actors: {selected.data}'
11             )
12
13         # If an AnimBP is open, get its nodes
14         if 'AnimBlueprint' in task or 'AnimGraph' in task:
15             nodes = self._ue5_bridge.get_anim_graph_nodes(
16                 self._current_asset_path
17             )
18             if nodes.success:
19                 context_parts.append(
20                     f'AnimGraph nodes: {nodes.data}'
21                 )
22
23         # Include UE5 context in the Claude prompt
24         ue5_context = '\n'.join(context_parts) if context_parts else 'UE5 not connected'
25
26         prompt = f'''Task: {task}
27 UE5 Editor State: {ue5_context}
28 What should the user do next?'''
29
30     return self._call_claude(screenshot, prompt)
```


IMPACT

Accuracy leap: Claude knows EXACTLY what's selected, what nodes exist, what properties are set — not guessing from pixels.

AnimGraph guidance: Claude can say 'connect the Locomotion output to the Final Animation Pose input' with CERTAINTY because it has the node list.

Property editing: Claude knows the current value and can say 'change

Mobility from Static to Movable' without reading small dropdown text.

Implementation: UE5Bridge already exists and is tested. Just wire it in.

Phase integration: Phase 2, new Task 8b (augment Claude call with UE5 data).

19. Enhancement: Progress Breadcrumbs

HIGH Progress Breadcrumbs

A semi-transparent sidebar on the overlay showing task progress. Steps auto-check as Claude detects completion. Critical for long workflows like AnimBP setup (20+ steps).

19.1 Breadcrumb Rendering

```

1 def paint_breadcrumbs(self, painter: QPainter, steps: list[dict]):
2     """Draw progress sidebar on the right edge of overlay."""
3     sidebar_w = 280
4     sidebar_x = self.width() - sidebar_w - 20
5     y = 60
6
7     # Semi-transparent background
8     painter.setBrush(QColor(10, 10, 20, 200))
9     painter.setPen(Qt.PenStyle.NoPen)
10    sidebar_h = len(steps) * 32 + 40
11    painter.drawRoundedRect(QRectF(
12        sidebar_x, y - 10, sidebar_w, sidebar_h), 10, 10)
13
14    for i, step in enumerate(steps):
15        step_y = y + i * 32
16        status = step.get('status', 'pending')
17
18        # Status icon
19        if status == 'completed':
20            painter.setPen(QColor(60, 220, 100)) # Green check
21            icon = '[x]'
22        elif status == 'current':
23            painter.setPen(QColor(0, 200, 255)) # Cyan arrow
24            icon = '[>]'
25        else:
26            painter.setPen(QColor(100, 100, 120)) # Gray
27            icon = '[ ]'
28
29        painter.setFont(QFont('Consolas', 10))
30        painter.drawText(sidebar_x + 12, step_y + 16, icon)
31
32        # Step text
33        color = QColor(220, 220, 240) if status != 'pending' else QColor(100, 100, 120)
34        painter.setPen(color)
35        painter.setFont(QFont('Segoe UI', 10))
36        text = step.get('label', f'Step {i+1}')
37        if len(text) > 30:
38            text = text[:28] + '..'
39        painter.drawText(sidebar_x + 50, step_y + 16, text)

```

19.2 Step Tracking in WatchAnalysis Schema

```
1 class StepProgress(BaseModel):
2     label: str
3     status: Literal['completed', 'current', 'pending']
4
5 class WatchAnalysis(BaseModel):
6     annotations: list[Annotation]
7     narration: str
8     total_steps: int
9     current_step: int
10    confidence: float
11    context: str
12    # NEW: step progress for breadcrumbs
13    steps: list[StepProgress] = []
```

IMPACT

UX critical: users always know where they are in a 20-step workflow.

Motivation: seeing checked steps creates momentum and confidence.

Error recovery: if Claude detects a regression, it un-checks the step.

Implementation: ~60 LOC in overlay.py + schema addition.

Phase integration: Phase 3 (Animation + TTS), new Task 3b.

20. Enhancement: 'Just Do It' Auto Mode Handoff

MED Auto Mode Handoff

The user sees a Watch Mode annotation (e.g., 'Click the Compile button') and decides they want Bionics to do it automatically. They click 'Just Do It' on the ControlPanel, and Bionics executes THAT ONE action via Auto Mode, then returns to Watch Mode.

20.1 Architecture

```

1 class WatchModeUI:
2     def __init__(self, watch_engine, executor, safety):
3         self.overlay = AnnotationOverlay()
4         self.controls = ControlPanel()
5         self._watch = watch_engine
6         self._executor = executor # From AgentCore
7         self._safety = safety
8
9         self.controls.do_it_clicked.connect(self._on_just_do_it)
10
11     def _on_just_do_it(self):
12         """Execute the current step via Auto Mode, then resume Watch."""
13         current = self._watch.current_analysis
14         if not current or current.confidence < 0.7:
15             return # Too uncertain to auto-execute
16
17         # Convert Watch annotation to Auto Mode action
18         action = self._annotation_to_action(current)
19         if action:
20             # Pause Watch Mode
21             self._watch.pause()
22             self.overlay.hide()
23
24             # Execute ONE action via Auto Mode executor
25             result = self._executor.execute(action)
26
27             # Resume Watch Mode
28             self.overlay.show()
29             self.overlay.apply_click_through()
30             self._watch.resume()
31
32     def _annotation_to_action(self, analysis):
33         """Convert a spotlight annotation to a click action."""
34         for ann in analysis.annotations:
35             if ann.type == 'spotlight' and ann.bbox:
36                 # Click the center of the spotlight
37                 cx = ann.bbox.x + ann.bbox.w // 2
38                 cy = ann.bbox.y + ann.bbox.h // 2
39                 return Action('click', {'x': cx, 'y': cy, 'button': 'left'})
40         return None

```

SAFETY NOTE

'Just Do It' routes through the existing SafetyLayer.
MODERATE actions (clicks, typing) require 1 confirmation.
DESTRUCTIVE actions (delete, overwrite) require 2 confirmations.
The user is always in the loop — this is guided automation, not blind execution.

IMPACT

Best of both worlds: learn with Watch Mode, automate with Auto Mode.
Seamless transition: one button press, zero context switching.
Reuses 100% of existing executor + safety infrastructure.
Implementation: ~50 LOC bridge between WatchEngine and ActionExecutor.

Phase integration: Phase 4 (Polish + GUI Integration), new Task 5b.

21. Enhancement: Zoom-on-Pin for Dense Graphs

MED Zoom-on-Pin

When Claude identifies a small or dense target (Blueprint pin, dropdown item, small button), Watch Mode auto-crops that region from the screenshot and re-sends it to Claude at full resolution. This solves the ~50px accuracy problem for packed AnimGraph node graphs.

21.1 Implementation

```

1 def zoom_analyze(self, screenshot, target_bbox, task):
2     """Re-analyze a cropped region at full resolution."""
3     x, y, w, h = target_bbox
4
5     # Add padding (50px each side)
6     pad = 50
7     x1 = max(0, x - pad)
8     y1 = max(0, y - pad)
9     x2 = min(screenshot.width, x + w + pad)
10    y2 = min(screenshot.height, y + h + pad)
11
12    # Crop – this region goes to Claude at FULL resolution
13    # No downscaling needed if region is < 1568px
14    cropped = screenshot.crop((x1, y1, x2, y2))
15
16    # Tell Claude the context
17    prompt = f'''This is a ZOOMED view of a small region.
18 The region is at ({x1},{y1})-({x2},{y2}) in the full screenshot.
19 Task: {task}
20 Identify the EXACT element to interact with.
21 Return coordinates relative to THIS cropped image.'''
22
23    analysis = self._call_claude(cropped, prompt)
24
25    # Map coordinates back to full screenshot space
26    for ann in analysis.annotations:
27        if ann.bbox:
28            ann.bbox.x += x1
29            ann.bbox.y += y1
30        if ann.position:
31            ann.position.x += x1
32            ann.position.y += y1
33
34    return analysis

```

21.2 When to Zoom

```
1 # Auto-zoom trigger in WatchEngine:
2
3 analysis = self._call_claude(screenshot, task)
4
5 # If confidence is low and a spotlight exists, try zooming
6 if analysis.confidence < 0.7:
7     for ann in analysis.annotations:
8         if ann.type == 'spotlight' and ann.bbox:
9             zoomed = self.zoom_analyze(
10                 screenshot,
11                 (ann.bbox.x, ann.bbox.y, ann.bbox.w, ann.bbox.h),
12                 task
13             )
14             if zoomed.confidence > analysis.confidence:
15                 analysis = zoomed # Use the zoomed version
16             break
```

IMPACT

Solves the accuracy problem for dense AnimGraphs and Blueprint editors.
Cropped region sent at FULL resolution — no API downscaling.
Auto-triggered only when confidence is low — no extra API cost when unnecessary.
No new dependencies: just PIL crop + existing Claude pipeline.
Implementation: ~40 LOC in watch_engine.py.

Phase integration: Phase 4, new Task 5c.

22. Updated Phase Map with Enhancements

All 6 enhancements integrated into the 4-phase plan:

Phase	Core Tasks	Enhancements	New LOC
P1: Core Overlay	Overlay window, QPainte	—	~400
P2: Claude Vision	WatchEngine, Pydantic s	SSIM gating (+8 LOC), D	~350
P3: Animation+TTS	Pulse glow, fade transi	Progress breadcrumbs (+	~200
P4: Polish+GUI	BionicsWindow integrati	'Just Do It' handoff (+	~250
TOTAL			~1,250

22.1 New File Summary (Updated)

File	LOC	Contains
core/watch_state.py	~20	WatchState enum + transitions
core/watch_engine.py	~350	WatchEngine + SSIM gating + di
gui/overlay.py	~500	AnnotationOverlay + ControlPan
gui/watch_panel.py	~120	WatchModePanel for BionicsWind
TOTAL NEW	~990	4 new files

22.2 Modified File Summary (Updated)

File	Changes	Lines Added
config.yaml	Add watch_mode: block	+15
gui/app.py	Add Watch Mode toggle, wire Wa	+80
TOTAL MODIFIED		+95

GRAND TOTAL

~1,085 lines of new/modified code across 6 files.
Zero new dependencies (QTextToSpeech ships with PyQt6).
Reuses 5 existing modules without modification.
4 phases, ~3-4 sessions to complete.

23. Testing Plan (Complete)

Test	Phase	Pass Criteria
Click-through test	P1	Click UE5 button THROUGH overl
Focus test	P1	Show overlay — UE5 retains key
Taskbar test	P1	Overlay not in taskbar or Alt+
Spotlight render	P1	Dim mask + cutout + glow borde
Arrow render	P1	Curved bezier arrow with arrow
Bubble render	P1	Dark rect + white text + step
ControlPanel click	P1	Buttons respond to clicks
SSIM gating	P2	No API call if screen unchange
Diff auto-highlight	P2	Green borders appear on change
Claude analysis	P2	Hotkey triggers capture + anal
Coordinate accuracy	P2	Spotlight within 30px of targe
Self-capture	P2	Screenshot does NOT contain ov
UE5 augmentation	P2	Claude prompt includes selecte
Pulse animation	P3	Smooth pulsing glow (no jitter
Fade transition	P3	Old annotations fade out, new
TTS narration	P3	Voice speaks narration clearly
TTS interrupt	P3	New analysis interrupts speech
Progress breadcrumbs	P3	Sidebar shows steps, auto-chec
GUI toggle	P4	Watch Mode button starts/stops
F8 hotkey	P4	F8 toggles Watch Mode globally
Multi-monitor	P4	Overlay on monitor with UE5
DPI scaling	P4	Annotations align at 125%/150%
Just Do It	P4	Single action executes then re
Zoom-on-pin	P4	Low-confidence re-analyzes at
API error	P4	Timeout shows error state, res
Clean shutdown	P4	Closing Bionics stops thread,

— End of Document —

Sworder:721 | Main Objective: AAA Studio Quality | 2026-03-29